

BDI Kernel Testing Guide

Overview

This document provides comprehensive guidance on testing the BDI kernel, including test categories, execution instructions, CI/CD integration, and profiling infrastructure.

Table of Contents

- [1. Test Categories](#)
- [2. Running Tests](#)
- [3. CI/CD Pipeline](#)
- [4. Stress Testing](#)
- [5. Profiling and Metrics](#)
- [6. Regression Testing](#)
- [7. Contributing Tests](#)

Test Categories

Unit Tests

Individual component tests that verify specific functionality in isolation.

Location: `bdi_kernel/tests/`

Components tested:

- Memory management (HAM, PMM, VMM)
- Scheduler primitives
- Process management
- Device drivers
- Syscall handlers

Run with:

```
cd bdi_kernel
./test_runner --category=unit
```

Integration Tests

Cross-component tests that verify interactions between subsystems.

Location: `bdi_kernel/tests/integration_tests.c`

Scenarios tested:

- Memory allocation with scheduler
- Process creation with device access
- IPC mechanisms
- System call flows

Run with:

```
./test_runner --category=integration
```

Stress Tests

High-load tests designed to find race conditions, memory leaks, and performance bottlenecks.

Location: `bdi_kernel/tests/stress/`

Test suites:

- **Memory Stress** (`memory_stress.c`): High-frequency allocation/deallocation, multi-threaded pressure, NUMA stress
- **Scheduler Stress** (`scheduler_stress.c`): Task creation storms, work stealing, context switch pressure
- **Process Stress** (`process_stress.c`): Fork bombs, rapid lifecycle, IPC stress

Run with:

```
# Run all stress tests for 60 seconds
./test_runner --category=stress --duration=60

# Run specific stress test
./tests/stress/memory_stress 3600 # 1 hour
./tests/stress/scheduler_stress 1800 # 30 minutes
./tests/stress/process_stress 900 # 15 minutes
```

Regression Tests

Tests for previously fixed bugs to prevent regressions.

Location: `bdi_kernel/tests/regression/`

Coverage:

- Bug #145: `krealloc` NULL pointer handling
- Bug #145: `ham_free` double-free protection
- Bug #146: Syscall registration order
- Edge cases: zero-size allocation, large allocations, alignment

Run with:

```
./test_runner --category=regression
# Or standalone
./tests/regression/regression_suite
```

Performance Tests

Benchmarks for measuring and tracking performance metrics.

Location: `bdi_kernel/tests/performance_tests.c`

Metrics:

- Memory allocation latency
- Context switch overhead

- Syscall latency
- Throughput measurements

Run with:

```
./test_runner --category=performance --output=json
```

Running Tests

Test Runner

The unified test runner provides a single interface for all test categories.

Basic usage:

```
cd bdi_kernel
make test # Build and run all tests

# Or use test runner directly
./test_runner [OPTIONS]
```

Options:

- -c, --category=CAT : Test category (unit|integration|stress|regression|performance|all)
- -f, --filter=PATTERN : Filter tests by name pattern
- -p, --parallel : Run tests in parallel
- -d, --duration=SEC : Duration for stress tests (seconds)
- -v, --verbose : Verbose output
- -o, --output=FORMAT : Output format (text|json|xml)
- -l, --list : List all tests
- -h, --help : Show help

Examples:

```
# Run all unit tests
./test_runner --category=unit

# Run memory-related tests with verbose output
./test_runner --filter=memory --verbose

# Run stress tests for 1 hour
./test_runner --category=stress --duration=3600

# List all available tests
./test_runner --list
```

Building Tests

```
cd bdi_kernel

# Build in debug mode (with sanitizers)
make BUILD_MODE=debug

# Build in release mode
make BUILD_MODE=release

# Build with specific compiler
make CC=clang-18

# Clean and rebuild
make clean && make
```

CI/CD Pipeline

GitHub Actions Workflows

The project uses GitHub Actions for continuous integration. Workflows are located in `.github/workflows/`.

Build & Test Workflow (`ci-build.yml`)

- **Trigger:** Push to main/develop, pull requests
- **Matrix:** Ubuntu 22.04/24.04 × GCC/Clang
- **Steps:** Build kernel, run basic tests, upload artifacts

Sanitizer Tests (`sanitizers.yml`)

- **Trigger:** Push to main/develop, pull requests
- **Sanitizers:** AddressSanitizer, UndefinedBehaviorSanitizer, MemorySanitizer
- **Purpose:** Detect memory errors, undefined behavior, uninitialized reads

Code Coverage (`coverage.yml`)

- **Trigger:** Push to main/develop, pull requests
- **Tools:** lcov, Codecov
- **Output:** Coverage reports uploaded to Codecov

Stress Tests (`stress-tests.yml`)

- **Trigger:** Daily at 2 AM UTC, manual dispatch
- **Duration:** 1 hour per test suite
- **Suites:** Memory, Scheduler, Process stress tests

Performance Benchmarks (`benchmarks.yml`)

- **Trigger:** Push to main, pull requests, manual dispatch
- **Output:** JSON benchmark results, historical comparison

Status Badges

Add these badges to your README:

```
! [CI Build] (https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvX-sEguG12Cy2rLnYl2IddLY\_RsBAHRrN8jiC51YcfmCJzxluBrkJML5op2jrJ8PQ2WairFQckunIDjBDGeJgEu8v-AkwUhzJglrckYqmWqQiSz72Y7j9hWwjrdx2iln7r5j331vUj63Z\_YVXaU/s1600/2020-01-27+19\_34\_18-ADF+PASS+CD+-+Release-41+-+Pipelines\_Copy.png)
! [Sanitizers] (https://i.ytimg.com/vi/jfL6I0VDgGw/hq720.jpg?sqp=-oaymwEhCK4-FEIDSFryq4qpAxMIARUAAAAAGAEIAADIQj0AgKJD&rs=A0n4CLCDIgyqNGN9bFR2zNmXseZ0xGqRGw)
! [Coverage] (https://codecov.io/gh/Mecca-Research/The-Binary-Decomposition-Interface/branch/main/graph/badge.svg)
```

Stress Testing

Memory Stress Tests

Purpose: Validate memory management under extreme load

Test patterns:

- High-frequency allocation/deallocation (16 threads)
- Memory pressure (large block allocations)
- Reallocation stress
- NUMA cross-node allocations

Metrics collected:

- Total allocations/deallocations
- Failed allocations
- Peak memory usage
- Allocation latency

Run standalone:

```
cd bdi_kernel/tests/stress
./memory_stress 3600 # Run for 1 hour
```

Scheduler Stress Tests

Purpose: Validate scheduler under high task load

Test patterns:

- Rapid task creation/destruction
- Work stealing scenarios
- Context switch storms
- Priority inversion detection

Metrics collected:

- Tasks created/completed
- Context switches
- Work stealing efficiency
- Completion rate

Run standalone:

```
cd bdi_kernel/tests/stress
./scheduler_stress 1800 # Run for 30 minutes
```

Process Stress Tests

Purpose: Validate process management under load

Test patterns:

- Controlled fork bombs
- Rapid process lifecycle
- IPC stress (pipes, signals)
- Resource limit enforcement

Metrics collected:

- Processes created/destroyed
- Fork failures
- IPC throughput

Run standalone:

```
cd bdi_kernel/tests/stress
./process_stress 900 # Run for 15 minutes
```

Profiling and Metrics

Profiler Infrastructure

The BDI kernel includes comprehensive profiling infrastructure for performance analysis.

Location: `bdi_kernel/profiling/`

Components:

- `profiler.c/h` : System-wide profiling
- `sched_stats.h` : Scheduler statistics
- `ham_metrics.h` : HAM allocator metrics

Using the Profiler

```
#include "profiling/profiler.h"

// Initialize profiler
profiler_config_t config = {
    .enabled = true,
    .collect_timestamps = true,
    .sampling_rate_hz = 1000,
    .output_file = "profile.json"
};
profiler_init(&config);

// Record events
profiler_start_event(PROFILE_MEMORY, "allocation");
// ... do work ...
profiler_end_event(PROFILE_MEMORY, "allocation");

// Increment counters
profiler_increment_counter("cache_hits");

// Export results
profiler_export_json("profile_results.json");
profiler_export_csv("profile_results.csv");

// Cleanup
profiler_shutdown();
```

Scheduler Statistics

Track scheduler performance metrics:

```
#include "profiling/sched_stats.h"

// Initialize
sched_stats_init();

// Record events
sched_stats_record_context_switch(cpu_id);
sched_stats_record_migration(from_cpu, to_cpu);
sched_stats_record_work_steal(thief_cpu, victim_cpu, success);

// Get statistics
cpu_sched_stats_t cpu_stats = sched_stats_get_cpu(0);
global_sched_stats_t global_stats = sched_stats_get_global();

// Export
sched_stats_export_json("sched_stats.json");
```

HAM Metrics

Monitor memory allocator performance:

```
#include "profiling/ham_metrics.h"

// Initialize
ham_metrics_init();

// Record events
ham_metrics_record_allocation(arena_id, size, success);
ham_metrics_record_deallocation(arena_id, size);
ham_metrics_record_cache_hit(arena_id);

// Get statistics
ham_global_stats_t stats = ham_metrics_get_global();
ham_numa_stats_t numa_stats = ham_metrics_get_numa(node_id);

// Export
ham_metrics_export_json("ham_metrics.json");
```

Regression Testing

Purpose

Prevent previously fixed bugs from reappearing.

Test Database

All regression tests are documented with:

- Bug ID
- Description
- PR where it was fixed
- Test function

Adding Regression Tests

1. Create test function in `bdi_kernel/tests/regression/regression_suite.c`:

```
static regression_result_t test_my_bug_fix(void) {
    // Test the previously buggy behavior
    // Return REGRESSION_PASS if fixed, REGRESSION_FAIL if regressed
    return REGRESSION_PASS;
}
```

1. Add to registry:

```
{
    .bug_id = "BUG-XXX",
    .description = "Description of bug",
    .fixed_in_pr = "#XXX",
    .test_func = test_my_bug_fix
}
```

1. Run regression suite:

```
./test_runner --category=regression
```


Contributing Tests

Guidelines

1. **Test Naming:** Use descriptive names that indicate what is being tested
2. **Documentation:** Add comments explaining the test purpose and expected behavior
3. **Assertions:** Use clear assertions with helpful error messages
4. **Cleanup:** Always clean up resources (memory, file descriptors, etc.)
5. **Isolation:** Tests should not depend on each other
6. **Determinism:** Tests should produce consistent results

Test Template

```
#include "test_framework.h"

static test_result_t test_my_feature(void) {
    // Setup
    void *resource = allocate_resource();
    if (!resource) {
        return TEST_RESULT_ERROR;
    }

    // Test
    bool result = my_feature(resource);

    // Verify
    if (!result) {
        cleanup_resource(resource);
        return TEST_RESULT_FAIL;
    }

    // Cleanup
    cleanup_resource(resource);
    return TEST_RESULT_PASS;
}

// Register test
REGISTER_TEST("my_feature", "Tests my feature functionality",
              TEST_CATEGORY_UNIT, test_my_feature)
```

Submitting Tests

1. Add tests to appropriate category directory
2. Update test runner if needed
3. Ensure tests pass locally
4. Submit PR with:
 - Test code
 - Documentation updates
 - CI workflow updates (if needed)

Troubleshooting

Common Issues

Tests fail with sanitizer errors:

- Check for memory leaks, use-after-free, or undefined behavior
- Run with `ASAN_OPTIONS=detect_leaks=1` for detailed output

Stress tests timeout:

- Reduce duration with `--duration` flag
- Check system resources (CPU, memory)

CI failures:

- Check workflow logs in GitHub Actions
- Reproduce locally with same compiler/OS
- Verify all dependencies are installed

Debug Mode

Build and run tests in debug mode for detailed output:

```
make BUILD_MODE=debug
./test_runner --verbose --category=unit
```

Performance Baseline

Track performance over time by comparing benchmark results:

```
# Run benchmarks
./test_runner --category=performance --output=json > baseline.json

# Compare with previous baseline
./scripts/compare_benchmarks.sh baseline.json current.json
```

Additional Resources

- [Kernel Architecture](#) (KERNEL_COMPLETION_PLAN.md)
- [Scheduler Overview](#) (SCHEDULING_SYSTEM_OVERVIEW.md)
- [Device Abstraction](#) (DEVICE_HARDWARE_ABSTRACTION.md)
- [System Services](#) (SYSTEM_SERVICES_IMPLEMENTATION.md)

Support

For questions or issues with testing:

- Open an issue on GitHub
- Check existing test documentation
- Review CI workflow logs